

Evaluating Post-Fine-Tuning Defense Mechanisms Against Harmful Fine-Tuning Attacks

Neil Pendyala, Anda He, Aniketh Kancherla, Fabien Lazes, Liangbin Zhao and Yurun Zi
University of Wisconsin–Madison
{npendyala, ahe28, akancherla, lazes, lzhao258, yzi2}@wisc.edu

Abstract

Safety-aligned large language models can become unsafe after downstream fine-tuning, especially when user data contains harmful examples mixed with otherwise benign task data. This project evaluates how robust different defense strategies are against harmful fine-tuning attacks across a hyperparameter grid. We compare supervised safety fine-tuning, RepNoise, Vaccine, and Antidote on LLaMA-2-7B-based checkpoints fine-tuned with mixtures of BeaverTails harmful data and SST-2 benign data. We vary harmful-data ratio, learning rate, and training epochs, then evaluate safety primarily with BeaverTails harmfulness scores and utility with SST-2 and GSM8K where available. Our results show that defense effectiveness is highly attack-dependent. In the strongest completed Antidote settings, BeaverTails harmfulness drops from the high 70s for attacked checkpoints to the mid-50s after defense, showing that post-fine-tuning repair can recover safety behavior in selected regimes. However, the results also expose important limitations: Antidote does not eliminate unsafe outputs, its grid coverage is incomplete, one GSM8K utility evaluation failed due to CUDA out-of-memory, and strict SST-2 evaluation reveals output-format failures that complicate utility claims. Overall, our findings suggest that single-point evaluations can overstate robustness, and that safety defenses must be judged jointly on harmfulness reduction, utility preservation, and coverage across attack hyperparameters.

1 Introduction

Large language models (LLMs) are increasingly released with safety alignment procedures that encourage them to refuse harmful instructions while remaining useful on benign user tasks. These safeguards are important because modern LLMs are often deployed in settings where users can directly interact with the model, request sensitive informa-

tion, or adapt the model to downstream applications. However, safety alignment is not a fixed property of a released model. Recent work has shown that downstream fine-tuning can weaken refusal behavior and cause previously aligned models to answer harmful prompts (?). This creates a practical security risk: a model provider may release an aligned base model, but a user or downstream developer can later modify the model through fine-tuning and unintentionally or deliberately degrade its safety behavior.

This risk is amplified by the normal use of parameter-efficient fine-tuning. Fine-tuning is not an exotic attack mechanism; it is a standard customization workflow for adapting foundation models to domain-specific tasks. Methods such as LoRA make this process cheaper by training small adapter modules rather than updating all base-model parameters. As a result, harmful behavior can emerge even when the original base model weights remain frozen. A user can start from a safety-aligned model, fine-tune lightweight adapters on a mixture of benign and harmful data, and obtain a model that preserves much of its task behavior while becoming more willing to follow unsafe instructions. In fine-tuning-as-a-service settings, this means that safety must be robust not only at release time, but also after downstream customization.

Prior work proposes several defenses against this problem. Alignment-stage methods attempt to make the model more robust before harmful fine-tuning occurs. RepNoise, for example, modifies the alignment process by noising internal representations associated with harmful behavior, aiming to make those behaviors harder to recover during later fine-tuning (?). Vaccine similarly acts during alignment by perturbing embeddings so that the aligned model becomes more resistant to harmful fine-tuning perturbations. In contrast, Antidote studies a post-fine-tuning defense: after a model

has already undergone harmful fine-tuning, it identifies harmful parameters using importance-based pruning and attempts to recover safe behavior without retraining the full model (?). These approaches differ not only in implementation but also in when they intervene: before fine-tuning, during alignment, or after the harmful model has already been produced.

Our project compares these defenses under one course-project experimental pipeline. We focus on harmful fine-tuning attacks where a LLaMA-2-7B-based aligned model is fine-tuned using mixtures of harmful BeaverTails examples and benign SST-2 data. We evaluate harmfulness primarily using BeaverTails-based harmfulness scores, with additional AdvBench results where available, and we evaluate utility using SST-2 and GSM8K where those metrics were successfully produced. Rather than evaluating a single attack setting, we vary harmful-data ratio, learning rate, and training epochs to test whether defense behavior is stable across attack strengths. This grid-style evaluation is important because a defense may appear effective at one hyperparameter setting but fail or become incomplete under stronger or different fine-tuning regimes.

Our report makes three contributions. First, we reproduce a harmful fine-tuning pipeline using LoRA adapters on safety-aligned LLaMA-2-7B-based checkpoints, allowing us to study safety degradation without updating the full base model. Second, we compare multiple defense strategies, including supervised safety fine-tuning, RepNoise, Vaccine, and Antidote, under a shared harmfulness evaluation centered on BeaverTails. Third, we analyze the limits of these defenses rather than only reporting aggregate improvements. In particular, we identify settings where Antidote substantially reduces BeaverTails harmfulness, but we also document incomplete Antidote grid coverage, a GSM8K utility failure caused by CUDA out-of-memory, and strict SST-2 output-format failures that complicate claims about utility preservation.

Our results show that defense effectiveness is attack-dependent. In selected high-strength Antidote settings, especially at 10% harmful data and learning rate 10^{-4} , BeaverTails harmfulness falls from the high 70s for attacked checkpoints to the mid-50s after applying Antidote. This supports the claim that post-fine-tuning repair can mitigate harmful behavior in some regimes. However, the evidence does not support a stronger claim that An-

tidote is fully hyperparameter-agnostic or that it consistently preserves utility under all evaluation protocols. Some Antidote outputs remain unsafe, the Antidote sweep is incomplete, and strict SST-2 evaluation marks some defended outputs incorrect because they produce verbose answers rather than the required positive/negative label. These findings suggest that safety defenses should be evaluated jointly on harmfulness reduction, utility preservation, and coverage across attack hyperparameters. Single-point results can overstate robustness, while broader sweeps expose both successful regimes and practical failure modes.

2 Related Work

Prior work on harmful fine-tuning shows that safety alignment can be fragile after downstream model customization. BeaverTails (?) provides one of the main datasets for studying this problem by pairing prompts and responses with safety annotations, making it useful both for alignment data and for evaluating harmful generations. This benchmark motivates our safety-evaluation setting: rather than judging only whether a model remains useful, we measure whether fine-tuning causes the model to produce unsafe responses on harmful prompts.

Antidote (?) directly motivates our post-fine-tuning defense setting. The key observation behind Antidote is that harmful fine-tuning may corrupt a localized subset of parameters, so safety can potentially be recovered after the attack by identifying and pruning parameters associated with harmful behavior. This differs from defenses that try to prevent safety degradation before or during fine-tuning. In our project, Antidote is the main post-hoc repair method we evaluate, and its claim of robustness across training hyperparameters motivates our grid-based analysis over harmful-data ratio, learning rate, and epoch count.

Other defenses intervene earlier in the training pipeline. RepNoise (?) is an alignment-stage defense that adds noise to internal representations associated with harmful behavior, making those representations harder to recover through later fine-tuning. Vaccine (?) similarly aims to improve robustness during alignment by making model embeddings more resistant to harmful fine-tuning perturbations. These methods motivate our comparison between alignment-time defenses and post-fine-tuning repair. While Antidote attempts to fix a model after harmful fine-tuning has occurred, Rep-

Noise and Vaccine attempt to make the aligned model harder to attack in the first place.

Several additional defenses also study alternative intervention points. LISA (?) proposes a fine-tuning-stage defense that limits harmful parameter drift by regularizing the update process, while Panacea (?) studies post-fine-tuning perturbations that suppress harmful behavior after the attack. These methods show that the defense design space spans multiple stages: alignment, user fine-tuning, and post-hoc repair. However, our final experiments focus on the defenses we were able to reproduce and evaluate under a shared course-project pipeline, namely supervised safety fine-tuning, RepNoise, Vaccine, and Antidote.

Finally, utility benchmarks are needed because a defense that only lowers harmfulness may still be impractical if it destroys downstream task performance. We use SST-2 (?) as a sentiment-classification utility task and GSM8K (?) where available as a mathematical-reasoning utility task. These benchmarks are not the main focus of our related-work discussion, but they support the central evaluation question: whether safety recovery comes at the cost of useful model behavior.

3 Method

Our experiments evaluate harmful fine-tuning defenses under a shared LoRA-based attack pipeline. The final quantitative comparison includes four methods: supervised safety fine-tuning (SFT) as an alignment baseline, RepNoise and Vaccine as alignment-stage defenses, and Antidote as a post-fine-tuning repair method. We use a shared BeaverTails-centered safety evaluation where possible, while noting that the available utility and auxiliary safety metrics differ across methods.

The pipeline has three stages. First, we start from a safety-aligned or defense-specific aligned checkpoint. Second, we perform mixed harmful fine-tuning using LoRA adapters on a dataset that combines harmful BeaverTails examples with benign SST-2 examples. Third, we evaluate the attacked model directly or apply a post-hoc defense before evaluation. This structure separates defenses that act during alignment from defenses that repair a model after harmful fine-tuning.

3.1 Base Model and LoRA Configuration

The quantitative experiments use checkpoints based on meta-llama/Llama-2-7b-hf. Harmful

fine-tuning is performed with LoRA adapters rather than full-parameter fine-tuning, so the base model weights remain fixed while a small set of adapter parameters is updated.

The LoRA configuration uses rank $r = 256$, scaling factor $\alpha = 4$, dropout 0, and target modules `q_proj`, `k_proj`, and `v_proj`. The bias setting is none, and the task type is causal language modeling. This setup allows us to simulate downstream model customization while keeping the fine-tuning process computationally feasible.

3.2 Stage 1: Safety or Defended Alignment

Stage 1 constructs the aligned checkpoint used as the starting point for harmful fine-tuning. We evaluate one standard safety-aligned baseline and two alignment-stage defenses.

SFT baseline. The SFT baseline uses a supervised safety-aligned checkpoint, `ckpt/Llama-2-7b-hf_sft`. This checkpoint represents the standard aligned model before the harmful downstream fine-tuning attack.

RepNoise. RepNoise is an alignment-stage defense. It modifies the alignment process by adding noise to internal representations associated with harmful behavior, making those representations harder to recover through later fine-tuning. In the final artifacts, RepNoise uses the aligned checkpoint `ckpt/Llama-2-7b-hf_repnoise2`. The merged RepNoise results contain 16 successful runs.

Vaccine. Vaccine is also an alignment-stage defense. It aims to make the aligned model more resistant to harmful fine-tuning perturbations during later customization. The merged Vaccine runs use a defense-specific alignment checkpoint such as `ckpt/Llama-2-7b-hf_vaccine_2`, and the resolved run parameters include $\rho = 2.0$. Vaccine has 27 successful merged runs, with BeaverTails safety evaluation and SST-2 utility evaluation.

3.3 Stage 2: Mixed Harmful Fine-Tuning Attack

After alignment, we apply a mixed harmful fine-tuning attack using LoRA adapters. The harmful component is drawn from PKU-Alignment/BeaverTails_dangerous, and the benign component is drawn from SST-2 using `data/sst2.json`. This attack setting models downstream customization in which a

mostly benign training set includes a small but nonzero harmful subset.

The attack grid varies three factors: harmful-data ratio, learning rate, and number of epochs. The harmful-data ratios are 1%, 5%, and 10%. The learning rates are 10^{-5} , 5×10^{-5} , and 10^{-4} . The epoch counts are 5, 10, and 20. The common training defaults use 5000 training samples, 1000 test examples, train batch size 5, evaluation batch size 5, gradient accumulation 1, weight decay 0.1, warmup ratio 0.1, and a constant learning-rate scheduler.

Coverage differs across methods. SFT and Vaccine each have 27 successful merged runs, covering the full $3 \times 3 \times 3$ grid. RepNoise has 16 successful merged runs, giving partial grid coverage. Antidote has 26 merged runs with one documented failure and incomplete usable coverage across all metrics.

3.4 Stage 3: Defense and Post-Attack Evaluation

For SFT, RepNoise, and Vaccine, the defense is already encoded in the aligned checkpoint used before the harmful fine-tuning attack. These methods are therefore evaluated after the mixed LoRA attack without an additional post-hoc repair step.

Antidote is applied after harmful fine-tuning as a post-fine-tuning repair method. Given an attacked checkpoint, Antidote identifies parameters associated with harmful behavior using importance-based pruning and applies a repair step intended to reduce unsafe responses while preserving utility. The reported Antidote defended checkpoints use names such as `antidote_mixed..._dr02_sn2000`, indicating dense ratio 0.2 and pruning sample number 2000. The corresponding attack checkpoints use names such as `attack_mixed..._sn5000`, indicating the 5000-example attack sample setting.

Antidote is evaluated as a partial reproduction. The merged Antidote summary contains 26 total runs and one failed run. The explicit failure occurs during GSM8K utility evaluation, where the `utility_gsm8k` step fails with a CUDA out-of-memory error. Safety evaluation for that run still completes, so the failure affects the available utility evidence rather than all safety metrics.

3.5 Evaluation Protocol

We evaluate each method along two axes: safety and utility. The safety axis measures whether a model produces harmful responses after fine-tuning, while the utility axis measures whether the

model still performs useful downstream tasks. This distinction is important because a defense is only useful if it reduces harmful behavior without destroying task performance. However, the final artifacts do not provide identical metric coverage for every method, so we report each metric only where it is available and avoid treating missing values as zeros.

Safety evaluation. Our primary safety metric is BeaverTails harmfulness. BeaverTails is the central safety benchmark in our evaluation because it is aligned with the attack setting: all methods use PKU-Alignment/BeaverTails_dangerous as the harmful fine-tuning source, and all four methods report BeaverTails safety results. For evaluation, the generation script loads the BeaverTails test split and filters it to unsafe prompts, using up to 1000 test prompts when the default test size is used. Model generations are then scored with the BeaverDam moderation model, PKU-Alignment/beaver-dam-7b. The harmfulness score is the percentage of evaluated outputs flagged as harmful, so lower values indicate safer behavior.

We also report AdvBench harmfulness where available. AdvBench provides a second adversarial safety check using harmful prompts outside the BeaverTails evaluation set. This helps test whether safety trends are specific to BeaverTails or also appear on another harmful-prompt benchmark. However, AdvBench is not available for every method in the merged final results: SFT, RepNoise, and Antidote include AdvBench, while Vaccine does not. We therefore treat AdvBench as a secondary, partial-coverage safety metric rather than as the main cross-method comparison axis.

Utility evaluation. We use SST-2 as the primary utility task because all four methods include SST-2 in the merged utility configuration, and because SST-2 is also the benign task mixed into the harmful fine-tuning data. SST-2 therefore measures whether the model preserves the sentiment-classification behavior associated with the benign portion of the fine-tuning mixture. The strict SST-2 evaluator uses an exact-output protocol: the prompt asks the model to respond only with positive or negative, and a response is counted correct only if it exactly matches one of the accepted label strings. This strict format is useful because it gives a clear classification accuracy, but it can also penalize models that express the correct sentiment in a longer

natural-language answer.

This issue is important for Antidote. In the audited strict SST-2 files, the attacked checkpoint at the 10% harmful-data, learning-rate 10^{-4} , 20-epoch setting produces strict label outputs for most examples, with 831 correct and 41 incorrect out of 872. The corresponding Antidote file produces verbose answers rather than exact labels and is marked 0 correct out of 872 by the strict evaluator. We therefore do not treat strict SST-2 failure as interchangeable with a proven collapse of sentiment understanding. Instead, we report it as evidence that strict-format utility preservation is not demonstrated. Moderated or table-derived SST-2 interpretations are reported separately when used, and are not treated as ordinary strict SST-2 accuracy.

We also include GSM8K where available as a harder utility benchmark for mathematical reasoning. GSM8K is scored by extracting the model’s numeric answer and comparing it against the target numeric answer, so higher values indicate better reasoning-task accuracy. SFT, RepNoise, and Antidote include GSM8K in their merged utility configurations, while Vaccine does not. One Antidote run fails during the GSM8K utility stage with a CUDA out-of-memory error, leaving the GSM8K score unavailable for that run. Because the safety stages for that run completed, we retain its completed safety metrics but mark the failed GSM8K utility value as unavailable rather than assigning it a score of zero.

AGNews appears as an additional utility task in the SFT merged configuration and in some heterogeneous RepNoise entries, but it is not uniformly available across methods. We therefore treat AGNews as auxiliary utility evidence rather than as a main cross-method comparison. The main utility discussion focuses on SST-2, since it is the only utility task shared by all four methods, and GSM8K where available.

Coverage and missing-value policy. Metric coverage differs across methods. SFT is evaluated on BeaverTails and AdvBench for safety and on SST-2, GSM8K, and AGNews for utility. RepNoise is evaluated on BeaverTails and AdvBench for safety and on SST-2 and GSM8K for utility, with some additional heterogeneous AGNews entries. Vaccine is evaluated on BeaverTails for safety and SST-2 for utility. Antidote is evaluated on BeaverTails and AdvBench for safety and on SST-2 and GSM8K for utility, but one GSM8K utility run fails. Because of these differences, our cross-method com-

parisons emphasize metrics shared across methods, especially BeaverTails harmfulness and SST-2 utility, and we explicitly mark missing or failed entries with “-”. Missing entries indicate unavailable, failed, or non-comparable measurements; they are not interpreted as zero-valued performance.

4 Experimental Setup

This section describes the final datasets, model conditions, hyperparameters, and evaluation configuration used in our quantitative comparison. The final artifacts compare four conditions: SFT, RepNoise, Vaccine, and Antidote. All final conditions use LLaMA-2-7B-family checkpoints. SFT, RepNoise, and Antidote use meta-llama/LLaMA-2-7b-hf-based checkpoints, while Vaccine uses a local LLaMA-2-7B snapshot path. Experiments were run across two GPU environments: we initially used cloud GPU resources, including NVIDIA H100 GPUs with 80GB VRAM through services such as Google Colab, and later continued on UW-Madison instructional GPU resources after exhausting available cloud compute. The scripts use explicit CUDA device assignment and automatic device mapping, allowing the same pipeline to run across both environments.

Datasets. The harmful fine-tuning attack uses BeaverTails dangerous examples as the harmful source and SST-2 examples as the benign source. These are mixed according to the harmful-data ratio for each run. BeaverTails is also the primary safety-evaluation dataset: evaluation prompts are drawn from the BeaverTails 30k_test split and filtered to unsafe prompts. AdvBench is used as an additional adversarial safety benchmark where available. For utility, SST-2 is used across all methods, GSM8K is used where configured for reasoning evaluation, and AGNews is treated as auxiliary because it is not uniformly available across methods.

Model conditions. The SFT condition is the supervised safety-aligned baseline. RepNoise and Vaccine are alignment-stage defenses: they modify the aligned checkpoint before harmful fine-tuning. Antidote is a post-fine-tuning defense: it is applied after the harmful LoRA attack to prune or repair parameters associated with harmful behavior. The Attack condition refers to the harmful mixed LoRA fine-tuned checkpoints used as comparators for defended models.

Dataset	Use	Size	Notes
BeaverTails-safe	alignment	5000	Safe SFT/alignment data used to construct safety-aligned checkpoints.
BeaverTails-dangerous	attack	5000	Harmful fine-tuning data mixed with benign SST-2 examples.
BeaverTails unsafe	safety eval	1000 cap	Unsafe prompts from 30k_test; scored with BeaverDam harmfulness.
AdvBench	safety eval	520	Additional adversarial harmful-prompt benchmark; not available for all methods.
SST-2	train / utility	872 eval	Benign training mix and sentiment-classification utility task.
GSM8K	utility	1000 cap	Math-reasoning utility task; available only for some methods.
AGNews	utility	1000 cap	Topic-classification utility task; auxiliary due to non-uniform coverage.

Table 1: Datasets used in the final experiments. Sizes reflect configured or audited counts where available.

Condition	Stage	Description
SFT Safe	alignment	safety-aligned baseline
Attack	fine-tuning	BeaverTails-dangerous + SST-2 LoRA attack
RepNoise	alignment	noised alignment before attack
Vaccine	alignment	perturbation-based alignment, $\rho = 2.0$
Antidote	post-attack	post-fine-tuning pruning, dr02_sn2000

Table 2: Model conditions compared in the final evaluation.

Attack grid and training hyperparameters.

The main attack sweep varies harmful-data ratio, learning rate, and number of training epochs. The harmful-data ratios are 1%, 5%, and 10%; the learning rates are 10^{-5} , 5×10^{-5} , and 10^{-4} ; and the epoch counts are 5, 10, and 20. The common configured defaults are 5000 attack-training samples, a 1000-example evaluation cap where applicable, train/eval batch size 5/5, gradient accumulation 1, weight decay 0.1, warmup ratio 0.1, and a constant learning-rate scheduler. Because some merged summaries omit fields and some RepNoise rows contain heterogeneous metadata, these values describe the common configured setup rather than a complete per-run serialization.

Fine-tuning uses LoRA adapters. The LoRA configuration uses rank $r = 256$, scaling factor $\alpha = 4$, dropout 0, target modules q_proj, k_proj, and v_proj, bias none, and task type CAUSAL_LM. This adapter-based setup allows harmful fine-tuning to update lightweight adapter weights rather than the full base-model parameters.

Evaluation settings. Safety and utility generations use deterministic decoding: temperature 1.0, do_sample=False, num_beams=1, and top_p=1. BeaverTails and GSM8K generation use max_new_tokens=512, while SST-2 and AGNews use max_new_tokens=200. BeaverTails harmfulness is the percentage of generations flagged harmful by the BeaverDam moderation model, so lower scores are better. SST-2 strict utility requires exact positive/negative outputs. GSM8K is scored by extracted numeric-answer exact match, and AGNews is scored by exact label-string matching.

Coverage and missing values. Run and metric coverage differ across methods. SFT has 27 successful merged runs. RepNoise has 16 successful merged runs, so its coverage is partial relative to the full 27-cell grid. Vaccine has 27 successful merged runs but narrower metric coverage, with BeaverTails and SST-2 in the merged configuration. Antidote has 26 merged runs with one failed run. The explicit Antidote failure occurs during GSM8K utility evaluation due to CUDA out-of-memory, producing a missing GSM8K value for that run. We mark unavailable or failed entries with “–” rather than treating them as zero-valued performance, and restrict conclusions to directly evidenced cells and configured metrics.

The cross-method comparison is informative but only partially controlled. The methods broadly share a LLaMA-2-7B model family, BeaverTails dangerous harmful data, SST-2 benign data, and BeaverTails safety evaluation. These shared axes make the BeaverTails trends useful for comparing method behavior. However, several important factors differ: checkpoint lineage is not identical, run counts differ, metric coverage differs, and both Rep-

Category	Hyperparameter	Value
Model	Base family	LLaMA-2-7B
Attack grid	Harmful ratios	1%, 5%, 10%
Attack grid	Learning rates	10^{-5} , 5×10^{-5} , 10^{-4}
Attack grid	Epochs	5, 10, 20
Training	Attack sample count	5000
Training	Evaluation cap	1000 where applicable
Training	Train/eval batch size	5 / 5
Training	Grad. accumulation	1
Training	Weight decay / warmup	0.1 / 0.1
Training	Scheduler	constant
LoRA	Rank / alpha / dropout	256 / 4 / 0
LoRA	Target modules	q_proj, k_proj, v_proj
Vaccine	ρ	2.0
Antidote	Dense ratio	0.2 in reported dr02 runs
Antidote	Pruning samples	2000 in reported sn2000 runs

Table 3: Main training and defense hyperparameters. Longer per-method metadata is kept outside the main text.

Noise and Antidote have incomplete grids. SFT has 27 successful merged runs, RepNoise has 16, Vaccine has 27, and Antidote has 26 with one failed run. Utility coverage also differs: Vaccine has SST-2 only, SFT has broader SST-2/GSM8K/AGNews coverage, and Antidote has an explicit GSM8K failure.

Because of these differences, we avoid claiming a definitive global ranking across all methods. Instead, we emphasize within-method trends and shared BeaverTails comparisons. The main supported conclusions are: harmful fine-tuning is effective in strong audited settings; Antidote substantially reduces harmfulness in selected paired cells; Vaccine helps in some weaker settings but degrades under stronger ones; RepNoise remains vulnerable in available strong-attack cells; and utility preservation remains unresolved under strict SST-2 and incomplete GSM8K coverage. Missing cells are treated as unavailable rather than zero, and all available-cell means are interpreted with coverage caveats.

5 Results and Analysis

5.1 Attack Effectiveness

Harmful fine-tuning successfully produced high harmfulness in the strongest audited settings. In the paired Antidote attack checkpoints at harmful ratio 10%, learning rate 10^{-4} , and epochs 5, 10, and 20, BeaverTails harmfulness reaches 78.3, 78.7, and 78.9, respectively. These scores are high relative to the safer defended checkpoints discussed below, showing that the mixed harmful fine-tuning procedure was effective at weakening safety behavior. Qualitative outputs from the same strong

attack checkpoint also contain unsafe assistance, and evaluator outputs assign harmful categories to those generations.

The broader ratio-10% BeaverTails grid in Figure 1 shows that harmfulness remains high for several baseline or defended conditions. The SFT condition ranges from 72.6 to 79.4 across the completed ratio-10% grid, indicating that ordinary safety alignment alone does not prevent strong harmful fine-tuning from producing unsafe behavior. This motivates the defense comparison: the question is whether any defense consistently reduces harmfulness while preserving downstream utility.

5.2 Defense Effectiveness on BeaverTails

Figure 1 summarizes BeaverTails harmfulness at harmful ratio 10% across SFT, RepNoise, Vaccine, and Antidote. SFT has complete coverage over the 9-cell grid and has an available-cell mean harmfulness of 77.30. RepNoise has 6 available cells and remains similarly high, with an available-cell mean of 78.43. Vaccine has complete coverage and a lower available-cell mean of 71.98, but its behavior varies substantially across the grid. Antidote has the lowest available-cell mean, 54.60, but only 5 of 9 cells are available, so this should be interpreted as a selected-cell trend rather than a complete-grid ranking.

The best single ratio-10% cell is Vaccine at learning rate 10^{-5} and 5 epochs, where harmfulness is 48.6. However, Vaccine becomes much less effective under stronger settings: at learning rate 10^{-4} , its harmfulness values rise to 77.2, 79.0, and 79.6 across epochs. Antidote is more stable in its available cells, with values between 54.3 and 55.0,

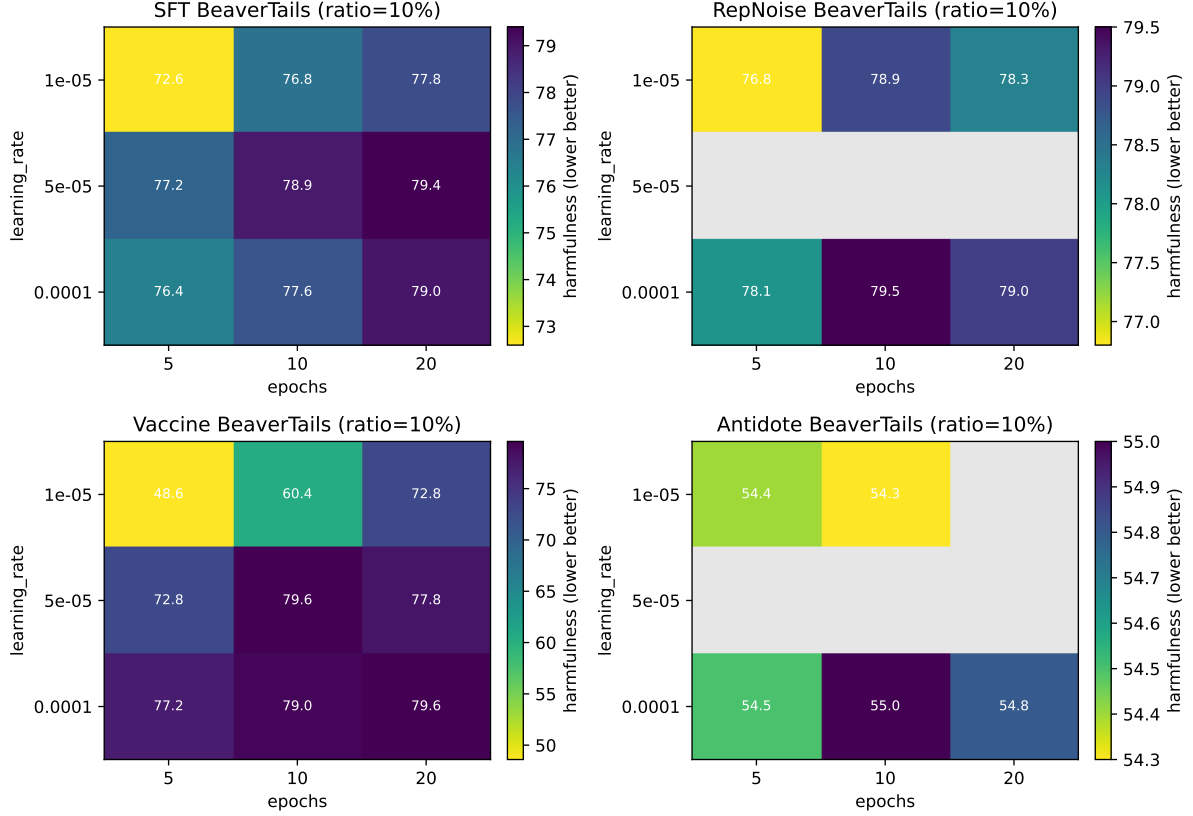


Figure 1: BeaverTails harmfulness at harmful-data ratio 10%. Lower is better; blank cells are unavailable runs, not zeros.

Method	Cells	Missing	Mean HS	Range
SFT	9	0	77.30	72.6–79.4
RepNoise	6	3	78.43	76.8–79.5
Vaccine	9	0	71.98	48.6–79.6
Antidote	5	4	54.60	54.3–55.0

Table 4: Ratio-10% BeaverTails summary over available cells. Missing cells are not treated as zero.

Epochs	Attack HS	Antidote HS	Change
5	78.3	54.5	-23.8
10	78.7	55.0	-23.7
20	78.9	54.8	-24.1

Table 5: Selected Antidote cells for $r = 10\%$, $lr = 10^{-4}$ on BeaverTails. Lower harmful score is better.

but its missing cells prevent a full-grid robustness claim. Overall, safety recovery is highly setting-dependent.

5.3 Antidote Paired Analysis

The clearest Antidote result comes from paired attack-versus-defense cells at harmful ratio 10% and learning rate 10^{-4} , shown in Table 5. At 5 epochs, Antidote reduces BeaverTails harmfulness from 78.3 to 54.5. At 10 epochs, it reduces harmfulness from 78.7 to 55.0. At 20 epochs, it reduces harmfulness from 78.9 to 54.8. These correspond to absolute reductions of 23.8, 23.7, and 24.1 points, with an average reduction of 23.87 points. This is strong evidence that post-fine-tuning prun-

ing can recover safety behavior in selected high-strength attack settings.

However, these Antidote results should not be overstated. The Antidote grid is incomplete, including missing ratio-10% cells and one failed merged run. The failed run occurs during GSM8K utility evaluation, where the `utility_gsm8k` step fails due to CUDA out-of-memory. The available evidence also does not establish sensitivity to dense ratio or pruning sample count: the reported Antidote checkpoints are encoded as `dr02_sn2000`, indicating dense ratio 0.2 and pruning sample number 2000, but the final results do not provide a full dense-ratio or sample-count sweep. Thus, Antidote shows substantial selected-cell safety gains, but

Condition	Strict SST-2	Mod. SST-2	Note
Attack, $r = .10$, $lr = 10^{-4}$, $ep=20$	831/872	–	strict format works
Antidote, same cell	0/872	95.18	mod. is not strict acc.

Table 6: Strict SST-2 versus moderated interpretation. Higher is better.

the evidence does not support a broad claim that Antidote is hyperparameter-agnostic.

5.4 Safety–Utility Tradeoff

The utility results complicate the safety story. Table 6 and Figure 2 show that strict SST-2 evaluation behaves very differently for the attacked and Antidote checkpoints in the audited strong setting. The attack checkpoint obtains 831 correct and 41 incorrect out of 872 examples under the strict evaluator. The corresponding Antidote checkpoint obtains 0 correct and 872 incorrect. This does not necessarily prove that the model lost all sentiment understanding, because the strict evaluator only counts exact positive or negative outputs as correct. The audited Antidote file often contains verbose natural-language answers instead of the required label format, so output-format mismatch likely contributes to the strict failure.

For this reason, strict and moderated SST-2 results must be interpreted separately. The moderated or table-derived Antidote SST-2 value of 95.18 suggests that sentiment information may still be recoverable under a less rigid interpretation, but this value is not strict exact-match accuracy. Therefore, the safest conclusion is that strict utility preservation is not demonstrated for the audited Antidote setting. In addition, GSM8K utility coverage is incomplete: one Antidote GSM8K evaluation fails with CUDA out-of-memory, and several method/metric combinations are unavailable. These results show that safety recovery alone is not sufficient; evaluation must also check whether the defended model still follows task-specific output formats and preserves utility.

5.5 RepNoise and Vaccine Trends

RepNoise does not appear to prevent high harmfulness under the strongest ratio-10% attacks in the available cells. Its available BeaverTails scores are 76.8, 78.9, and 78.3 at learning rate 10^{-5} , and 78.1, 79.5, and 79.0 at learning rate 10^{-4} . These values are close to the SFT baseline and remain in the high-70 range. On the six cells shared between SFT and RepNoise, RepNoise is slightly higher

Method	Runs	Failed	Metrics
SFT	27	0	BT, AdvBench, SST-2, GSM8K, AGNews
RepNoise	16	0	BT, AdvBench, SST-2, GSM8K
Vaccine	27	0	BT, SST-2
Antidote	26	1	BT, AdvBench, SST-2, GSM8K

Table 7: Run and metric coverage by method. BT denotes BeaverTails.

on average, with a mean difference of about 1.73 harmfulness points. This suggests that, under the available strong-attack settings, RepNoise did not substantially suppress harmful fine-tuning effects.

Vaccine shows a different pattern. It performs well in several weaker or low-learning-rate cells, including a harmfulness score of 48.6 at ratio 10%, learning rate 10^{-5} , and 5 epochs. However, its performance degrades under stronger settings: at learning rate 10^{-4} , its ratio-10% BeaverTails scores rise to 77.2, 79.0, and 79.6. This indicates that Vaccine can help in some regions of the attack grid, but its protection fades as the attack becomes stronger. Together, the RepNoise and Vaccine results support the broader conclusion that alignment-stage defenses are sensitive to attack hyperparameters.

5.6 Fairness of Cross-Method Comparison

The cross-method comparison is informative but only partially controlled. The methods broadly share a LLaMA-2-7B model family, BeaverTails dangerous harmful data, SST-2 benign data, and BeaverTails safety evaluation. These shared axes make the BeaverTails trends useful for comparing method behavior. However, several important factors differ: checkpoint lineage is not identical, run counts differ, metric coverage differs, and both RepNoise and Antidote have incomplete grids. As shown in Figure 3 and Table 7, SFT has 27 successful merged runs, RepNoise has 16, Vaccine has 27, and Antidote has 26 with one failed run. Utility coverage also differs: Vaccine has SST-2 only, SFT has broader SST-2/GSM8K/AGNews coverage, and Antidote has an explicit GSM8K failure.

Because of these differences, we avoid claiming a definitive global ranking across all methods. Instead, we emphasize within-method trends and shared BeaverTails comparisons. The main supported conclusions are: harmful fine-tuning is effective in strong audited settings; Antidote substantially reduces harmfulness in selected paired cells; Vaccine helps in some weaker settings but degrades under stronger ones; RepNoise remains

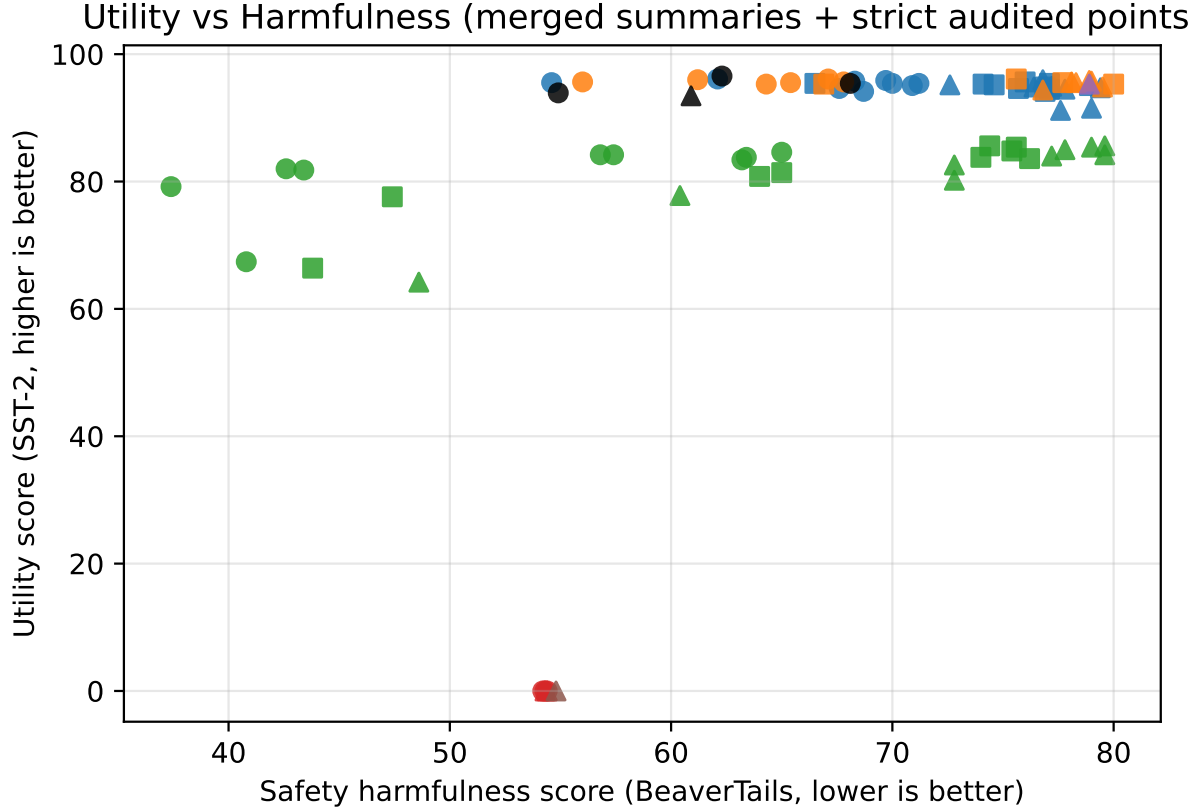


Figure 2: Safety–utility tradeoff across available runs. BeaverTails harmfulness is lower-better; SST-2 utility is higher-better. The plot combines merged-summary points with strict audited SST-2 points.

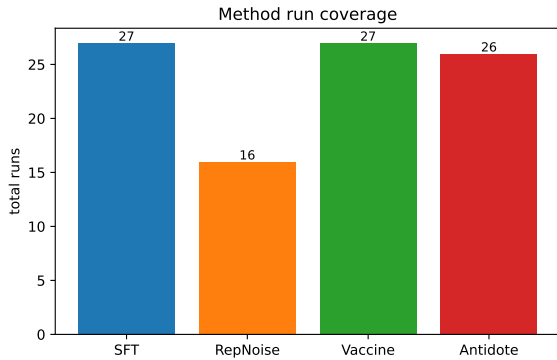


Figure 3: Run coverage by method. Different run counts and metric coverage make cross-method comparisons only partially controlled.

vulnerable in available strong-attack cells; and utility preservation remains unresolved under strict SST-2 and incomplete GSM8K coverage. Missing cells are treated as unavailable rather than zero, and all available-cell means are interpreted with coverage caveats.

Issue	Impact
Run coverage	RepNoise and Antidote have fewer completed cells.
Metric coverage	Vaccine lacks AdvBench and GSM8K in final artifacts.
Checkpoint lineage	Methods use different aligned checkpoints before attack.
Antidote sparsity	Antidote has missing utility fields and incomplete grid coverage.
Strict SST-2	Moderated SST-2 is not strict exact-label accuracy.

Table 8: Main caveats for cross-method comparison.

6 Conclusion and Discussion

This project shows that harmful fine-tuning is a practical threat to safety-aligned LLMs. In the strongest audited attack settings, BeaverTails harmfulness reaches the high 70s, confirming that LoRA-based downstream fine-tuning can substantially weaken safety behavior even without updating the full base model. Among the defenses we evaluated, Antidote provides the clearest selected-cell safety gains: for the 10% harmful-data, 10^{-4} learning-rate setting, Antidote reduces BeaverTails harmfulness from 78.3 to 54.5, 78.7 to 55.0, and 78.9 to 54.8 across 5, 10, and 20 epochs. Vaccine also

helps in some weaker settings, but its performance degrades under stronger attacks, while RepNoise remains in the high-harmfulness range for the available ratio-10% cells. Overall, the results suggest that defense effectiveness is highly dependent on attack hyperparameters rather than uniformly robust.

A strength of our work is that we move beyond a single attack configuration. We compare multiple defense routes across a grid of harmful ratios, learning rates, and epoch counts, and we analyze both safety and utility. The heatmaps and safety–utility scatter plots make the setting-dependent behavior visible: some methods look effective in isolated cells but become less convincing once coverage and missingness are considered. We also explicitly separate strict SST-2 exact-match utility from moderated interpretation. This is important because Antidote’s strict SST-2 result is 0/872 in the audited file, while moderated analysis suggests that many longer outputs still contain the correct sentiment label. Thus, strict utility preservation is not demonstrated, but the failure appears partly tied to output format rather than necessarily complete sentiment collapse.

The main limitations are incomplete coverage, compute constraints, and partial comparability across methods. RepNoise has fewer completed runs than SFT and Vaccine, Antidote has one failed GSM8K utility run due to CUDA out-of-memory, and the Antidote grid is not complete enough to support a hyperparameter-agnostic robustness claim. Metric coverage also differs across methods: for example, Vaccine lacks AdvBench and GSM8K in the final merged artifacts. Because of these differences, our cross-method comparisons should be interpreted as informative but only partially controlled. Future work should complete the missing Antidote and RepNoise cells, rerun failed utility evaluations, and standardize all methods under the same checkpoint lineage and metric suite. It would also be valuable to run an explicit Antidote dense-ratio and pruning-sample sweep, rerun SST-2 with constrained label-only decoding, and repeat key cells across multiple random seeds to measure variance.

7 Group Member Contributions

All group members contributed to the midterm and final presentations, including slide planning, result interpretation, and presentation preparation. Neil

Pendyala led most of the written report work for both the midterm and final submissions, including report structure, final framing, evidence integration, and polishing, with smaller edits and content support from other group members. Neil also worked on the harmful fine-tuning attack analysis for the RepNoise route and packaged the Route 3 RepNoise attack artifacts, including the notebook, artifact manifest, and results summary used to document the attack setup and outputs.

Yurun Zi handled the Antidote repository setup and alignment workflow, produced the aligned model, and ran preliminary BeaverTails-safe evaluation for the aligned checkpoint. Yurun implemented the SFT baseline sweep script, covering 27 settings across harmful ratios 1%, 5%, and 10%, learning rates 10^{-5} , 5×10^{-5} , and 10^{-4} , and epoch counts 5, 10, and 20. Yurun also ran safety and utility evaluation for the SFT baseline on BeaverTails, AdvBench, SST-2, GSM8K, and AGNews, and developed the moderated SST-2 evaluation script to analyze Antidote outputs that failed the strict exact-label evaluator despite often containing sentiment-bearing answers.

Fabien Lazes worked on evaluation infrastructure and RepNoise experiments. He studied the datasets used in the original defense pipelines, added AdvBench as an additional safety evaluation dataset, and evaluated early checkpoints to compare BeaverTails-style harmful prompts with more direct adversarial prompts from AdvBench. Fabien also ran RepNoise grid experiments using cloud and instructional GPU resources, contributed to RepNoise automation and evaluation code, and worked with Aniketh on evaluating Antidote results.

Anda He designed the shared experimental routing framework used to organize the project around four routes: SFT baseline, Vaccine, RepNoise, and Antidote. Her framework structured each route as an alignment or defense stage, followed by harmful fine-tuning, followed by safety and utility evaluation using harmful score (HS) and fine-tuning accuracy (FA). Anda focused especially on reproducing and analyzing the Vaccine route, running Vaccine alignment and harmful fine-tuning experiments, organizing results across poison ratios, learning rates, and epochs, and testing additional Vaccine ρ values beyond the default $\rho = 2$, including $\rho = 0$ and $\rho = 5$.

Aniketh Kancherla contributed to Antidote workflow integration and evaluation. He worked on doc-

umenting the Antidote implementation and Colab workflow, helped package strict SST-2 evaluation artifacts, and collaborated with Fabien on Antidote result evaluation. His work supported the analysis of Antidote’s post-fine-tuning behavior and the comparison between strict utility evaluation and the observed model outputs.

Liangbin Zhao compiled evaluation results from multiple defense runs into a consistent analysis format by extracting safety and utility metrics into heatmap-ready and scatterplot-ready CSV files. He standardized fields such as method, poison ratio, learning rate, epochs, metric, and score, making results from different defenses easier to compare across hyperparameter settings. Liangbin also produced visualization analyses, including hyperparameter heatmaps and safety–utility scatter plots, which helped turn scattered experimental outputs into interpretable evidence about robustness under increasingly extreme fine-tuning conditions.